Faculty of Electrical Engineering,
Computer Science and Information Technology Osijek
(FERIT Osijek)

Embedded Systems

Peugeot Infotainment Project

Author(s):
Mislav Milinković
Ivan Brajinović

August 31, 2025

Academic year 2024/2025

Contents

1 The project **2**
- 1.1 Intro . . . 2
- 1.2 Main Challenges . . . 2

2 VAN Bus **4**
- 2.1 Enhanced Manchester code . . . 4
- 2.2 Messages . . . 4
  - 2.2.1 Message types . . . 6
- 2.3 VAN network . . . 7

3 Gateway module hardware **9**
- 3.1 Communication hardware . . . 9
  - 3.1.1 TSS463C . . . 9
- 3.2 Voltage levels . . . 9
- 3.3 Connecting to the car . . . 9
- 3.4 ESP32 Flashing . . . 10
- 3.5 Power . . . 11
- 3.6 Raspberry Pi . . . 11
- 3.7 Misc . . . 11

4 Gateway module software **12**
- 4.1 Framework and approach . . . 12
- 4.2 Receiving VAN frames . . . 12
- 4.3 Parsing VAN frames . . . 13
- 4.4 Sending VAN frames . . . 13
- 4.5 UART communication . . . 14
- 4.6 Event handling and tasks . . . 14
- 4.7 Handling power states . . . 14
- 4.8 The ULP coprocessor . . . 14

5 Infotainment computer software **18**

1 The project

1.1 Intro

The goal of this project was to create a gateway module for an infotainment system that will be used in a Peugeot 206. The system itself has two major components: the gateway module and the infotainment computer. The gateway module is a custom board based on an ESP32 Wroom 32E module. The board handles power, communication with the car and communication with the infotainment computer. The infotainment computer is a Raspberry Pi 4 with a DSI touchscreen display and a custom Linux distribution. The computer handles the display and audio.

This project is focused primarily on the gateway module.

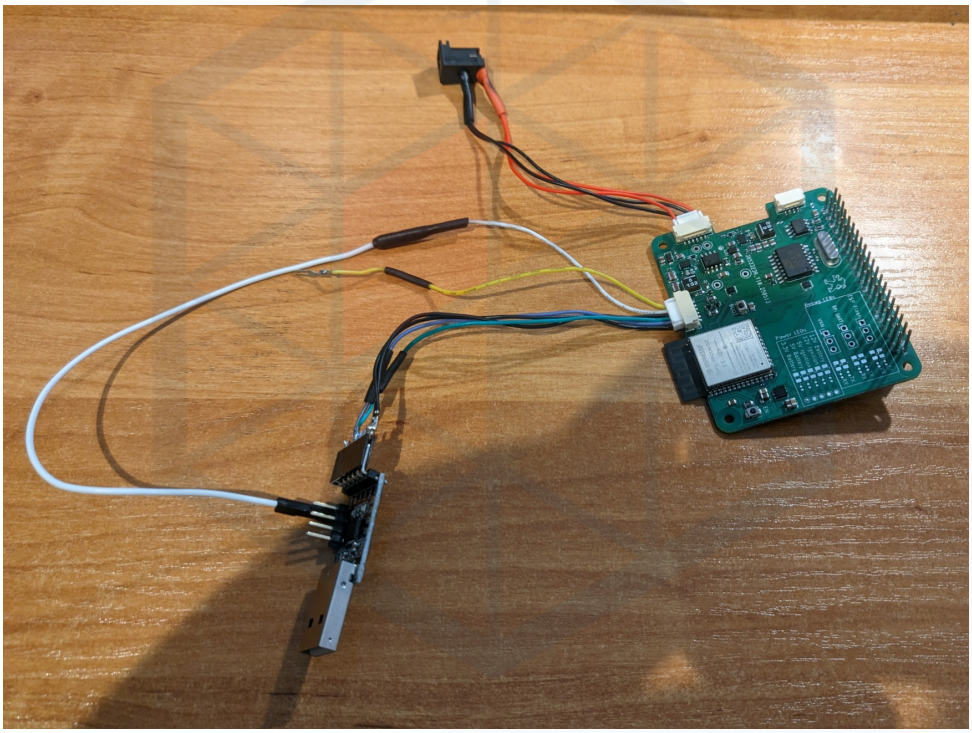

Figure 1: The gateway module setup for development

1.2 Main Challenges

This project has two, maybe three main challenges to consider. The obvious one is car communication. The Peugeot 206 doesn't use standard CAN bus communication, but rather something called VAN, see chapter 2. This communication standard is obsolete today, as it was only used in Peugeot and Citroën cars for a limited time. Because of that, almost no hardware exists than can understand the protocol, and about as much documentation. Thus, we must create our own parser for receiving VAN frames, and figure out how to send them as well.

The second challenge would be power. We have components requiring both 5 Volts and 3.3 Volts, as well as two power rails: a permanent source and a switched source. The other problem is **very** noisy and, sometimes, unstable input power, so cheap DC-DC converters will not cut it.

The last challenge, and maybe the most underestimated one, is writing a robust firmware for the gateway module itself. The module doesn't just need to parse the VAN frames, it needs to manage its own state, figure out the car state, handle the infotainment computer's power input and communication, handle sleep states and manage its own low power state, and, most importantly, not crash.

2 VAN Bus

VAN (Vehicle Area Network) is a serial communication bus developed by the PSA group (Peugeot, Citroën) and Renault as an altenative to the CAN bus. It's defined in the ISO standard **ISO 11519-3**. It utilizes the same physical layer as the CAN bus (a differential pair) and identical bus arbitration method, but it differs in the way the data is packaged.

Like CAN, the VAN protocol uses identifiers to differentiate between messages and perform bus arbitration. Additionally, it also has the ability for so called "In-frame" responses, bigger message sizes (28 bytes instead of 8 bytes in CAN) and the ability to directly communicate with another device.

Data on the VAN network is packed into frames. Frames consist of multiple parts: Start of frame (**SOF**), message identifier (**IDEN**), command (**COM**), message data (**DATA**), frame checksum (**FCS**), end of data signal (**EOD**), acknowledge signal (**ACK**) and the end of frame signal (**EOF**).

SOF	IDEN	Command (4 bit, 5 TS)				DATA max.	FCS	EOD	ACK	EOF
10 TS	12 bit					28 byte	15 bit	2 TS	2 TS	4 TS
	15 TS	EXT	RAK	R/W	RTR	280 TS	18 TS			

Table 1: VAN frame format

2.1 Enhanced Manchester code

All VAN frames are coded using Enhanced Manchester coding (e-Manchester). With e-Manchester, after sending 3 NRZ bits, the fourth bit is sent as a Manchester bit. Sending data this way ensures easier syncronization on the bus. This is a more predictable method of bitstuffing as opposed to the CAN bus method.

In Table 1 we can see the number of bits for each part of the frame, as well as the number of Time Slots (TS). Since a Manchester bit takes two time slots to send, 4 bits in a row encoded with e-Manchester takes 3 + 2 Time slots to send.

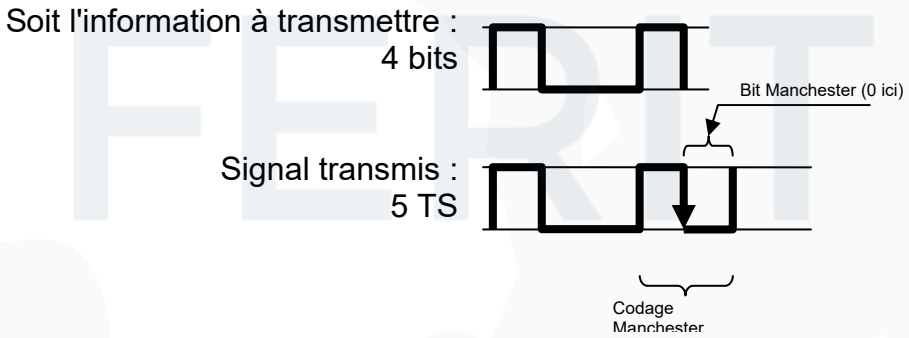

Figure 2: e-Mancherster encoding

2.2 Messages

A device/module connected to the network can be one of three types:

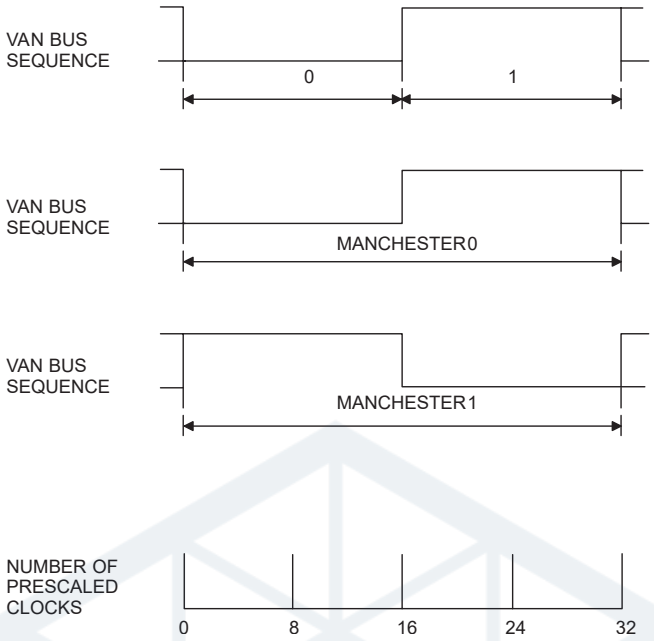

Figure 3: NRZ and Manchester bits.

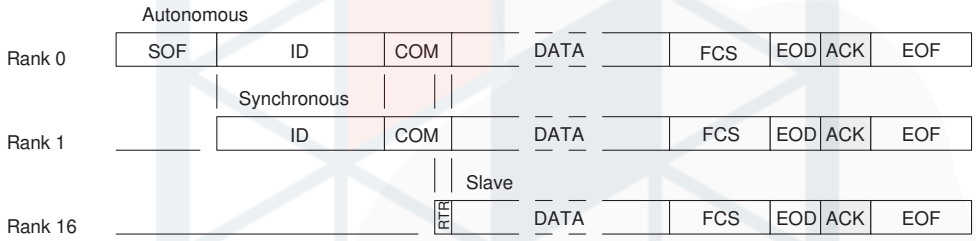

Figure 4: Module types

- The Autonomous module. A bus master. It can send **SOF** sequences, initiate data transfers and receive messages.
- The Synchronous access module. It cannot send **SOF** sequences, but can initiate data transfers and receive messages.
- The Slave module. It can only send messages using in-frame responses and can also receive messages.

The VAN protocol supports multiple message types. They are differentiated using the identifier (**IDEN**) and command (**COM**) fields. The identifier is 12 bits long and is followed by the 4 command bits. Due to the way the bus arbitration works, a smaller identifier has priority on the bus. While the identifier value is completely up to the user, the command bits are strictly defined and determine the message type that will be sent on the bus:

- EXT - always 1
- RAK - „Request Acknowledge“. If 1, the receipient must acknowledge the message reception. If 0, the ACK signal must not be sent.
- R/W - „Read/Write“. Signals the data direction. If 0, then it's a 'write' message, a device is sending data intended for another device. If 1, then it's a 'read' message, and it implies a request for data and expects an answer.

- RTR - „Remote Transmission Request“. If 0, the message contains data. If 1, the message contains no data and an in-frame response request is implied. The combination R/W = 0 and RTR = 1 is forbidded on the bus.

2.2.1 Message types

Here we will list frame types relevant to this project. There are more, but we only use these.

Normal frame The simplest frame type. One module sends an entire frame with data, and may or may not request an ACK. In the case no ACK is requested, the message is treated as a broadcast message.

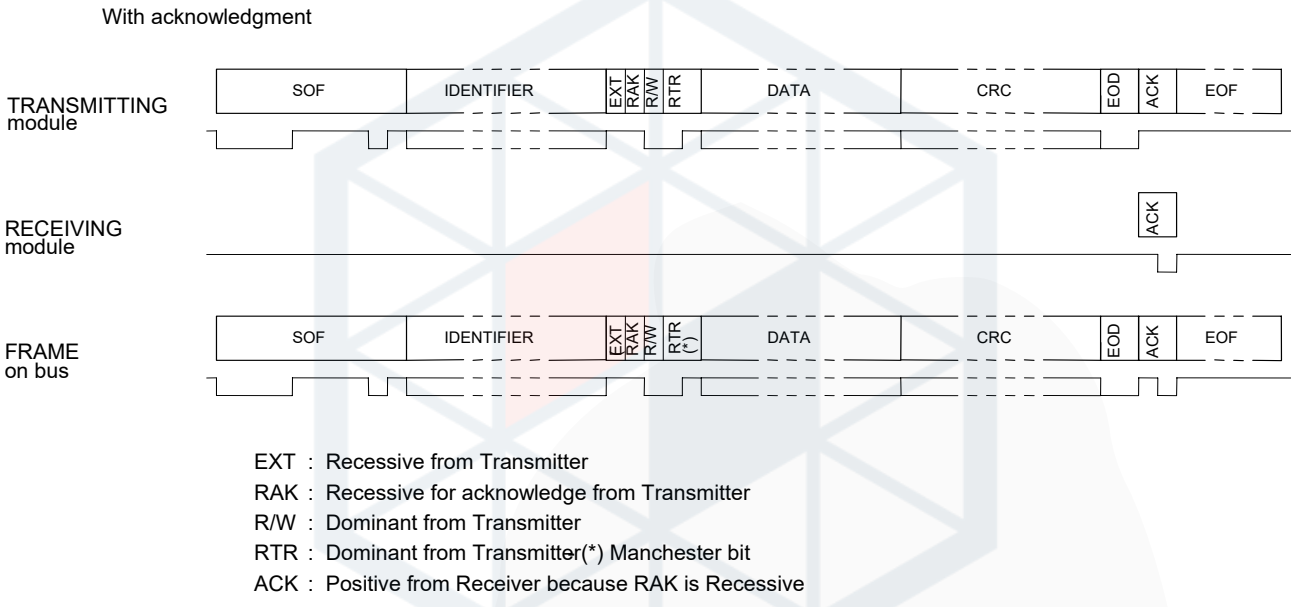

Figure 5: Normal frame transimssion with ACK

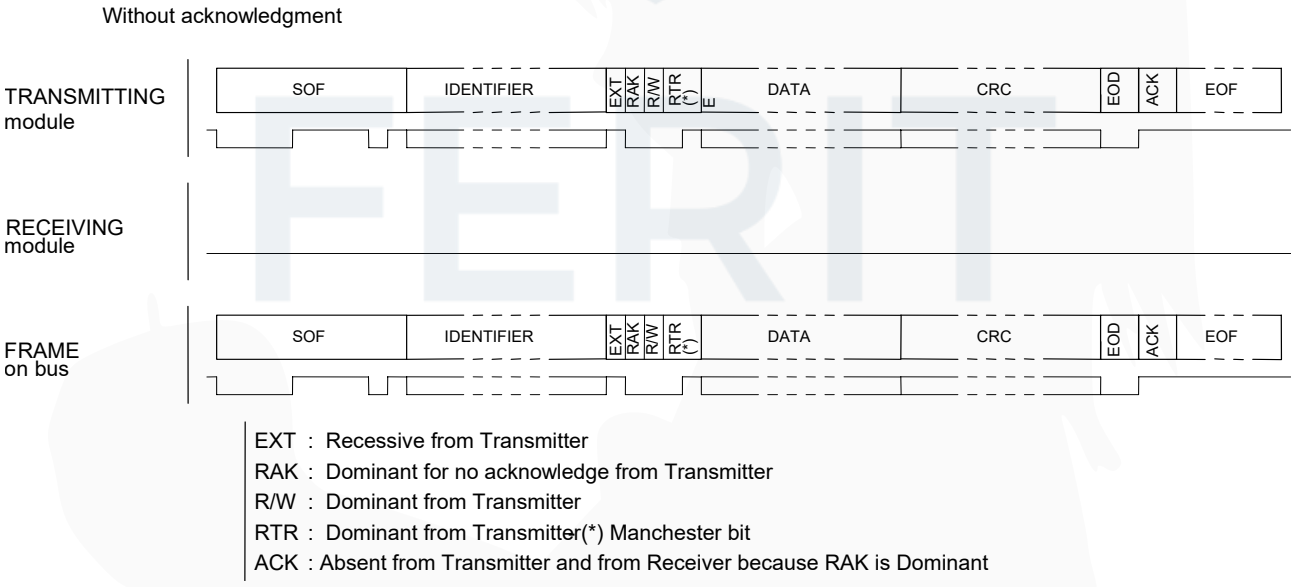

Figure 6: Normal frame transimssion without ACK

Reply Request Frame with Immediate Reply This is the only frame in which a slave module can transmit data. The only difference on the bus is the R/W bit changed state compared to the normal frame. The bus master generates the **SOF**, the initiator generates the identifier and first 3 bits of the command, and the ACK signal. The rest (RTR, data and checksum) are generated by the replying module.

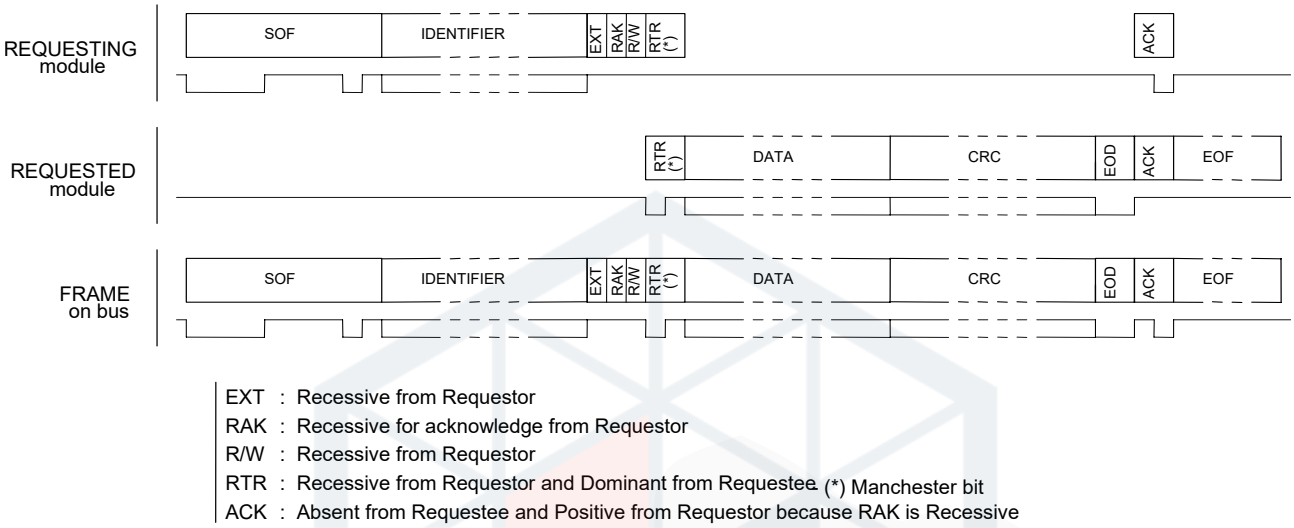

Figure 7: Reply Request Frame with Immediate Reply

2.3 VAN network

We can connect multiple modules together to create a network. We can also connect multiple networks together and use a gateway module to route data between networks. In the Peugeot 206 that we are building this project for, we have 1 CAN network *CAN Engine* andd 3 VAN networks: *VAN Comfort*, *VAN Body 1* and *VAN Body 2*.
We can see the layout in figure 8. From there we also see the gateway module, the BSI. This module is present in all PSA group cars. This particular BSI is from the AEE2001 generation, the only generation to use the VAN bus. Other generations, the AEE2004 and AEE2010 both exclusively use CAN.

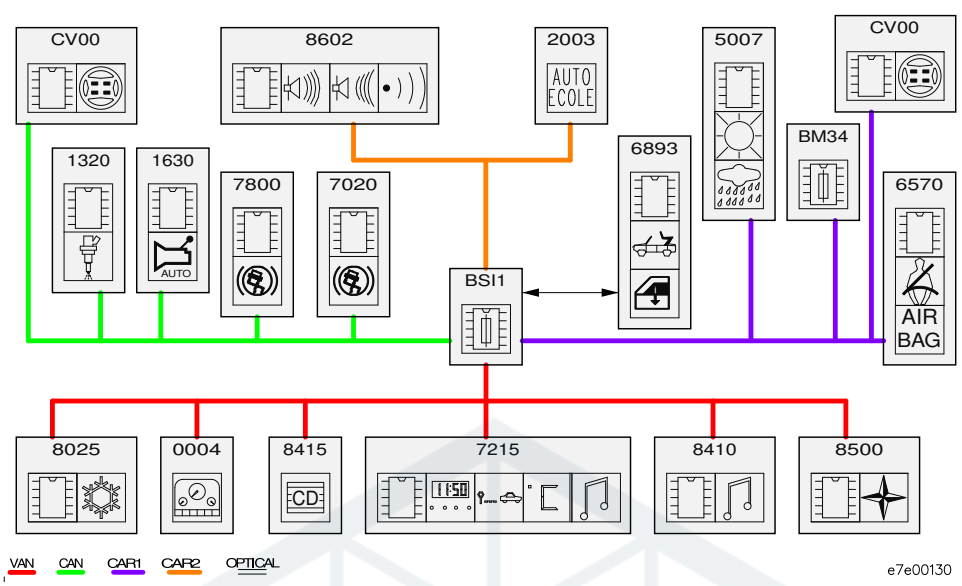

Figure 8: VAN and CAN Network layout in the Peugeot 206

Our infotainment system will be connected in place of the CD changer, module number 8415 on figure 8 on the *VAN Comfort* bus.

3 Gateway module hardware

The gateway module is designed as a Raspberry Pi HAT board. It has to contain necessary hardware for power regulation and car communication. It also has to expose all IO for connecting external devices such as an ESP32 programmer and a power cable for the Raspberry Pi display.

3.1 Communication hardware

In section 2 we have established that the VAN bus is a differential protocol much like the CAN bus and, as a result, we can use readily available CAN transciever hardware. However, we still need hardware that can generate valid VAN frames. For that we use the Atmel TSS463C VAN driver chip. It is the only VAN driver chip that exists. For the CAN transciever we use the MCP2551 because it was proven to work with early prototypes and, with it's 5 volt logic, integrates nicely with the TSS.

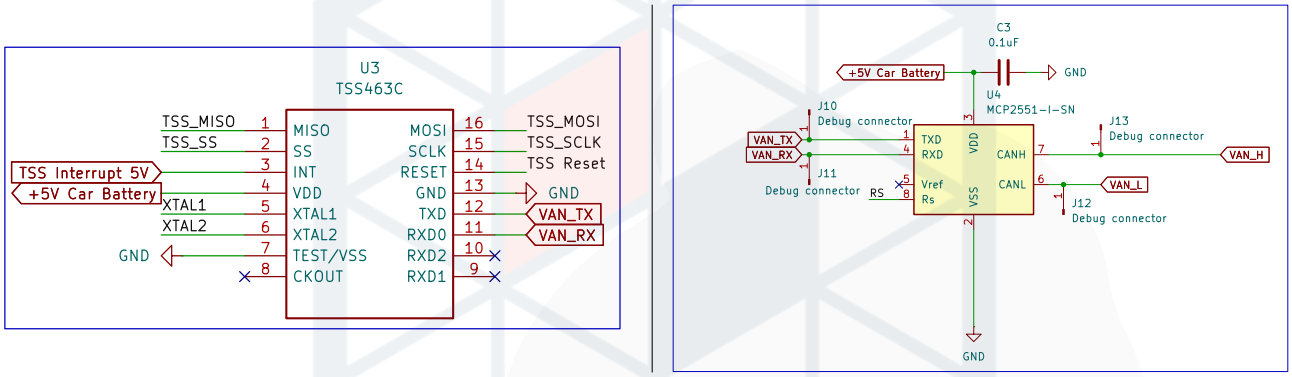

Figure 9: TSS463C and MCP2551 in the schematic

3.1.1 TSS463C

The TSS463C is the VAN driver chip we use for sending VAN frames to the car. It is the only standalone VAN driver chip that exists, as all other modules inside the car have a VAN driver built into their SOCs. The TSS uses SPI to communicate with the ESP32.

3.2 Voltage levels

The TSS463C and the MCP2551 both use 5V logic, while all other components on the board, including the ESP32, use 3.3V logic. For that we use a pair of NTS0104 level shifters. One of them is used for the TSS SPI communication with the ESP, while the other is used for connecting the VAN RX line from the MCP to the ESP32 for reading VAN frames, and the TSS interrupt pin.

3.3 Connecting to the car

We are connecting to the car in place of the CD Changer unit. The CD Changer connects to the original RD3 Headunit in the C3 (pins 13 to 20) portion of the ISO 10487 radio connector. See Figure 10 for the pinout.

Pins 13 to 17 for data and power are connected to the gateway module with a JST GH 6 pin connector, while pins 18 to 20 are wired to a 3.5mm audio cable and connected to the Raspberry Pi. Since the naming convention is different between Figures 10 and 11, see

No.	Description
1	N.C.
2	N.C.
3	N.C
4	N.C.
5	N.C.
6	N.C.
7	VOICE SYN
8	VOICE SYN-GND
9	N.C.
10	N.C.
11	AUX(+)
12	AUX(-)
13	VAN DATA
14	VAN DATA/
15	CD A/C GND
16	CD A/C B/U
17	+VAN
18	CD A/C S-GND
19	CD A/C L
20	CD A/C R

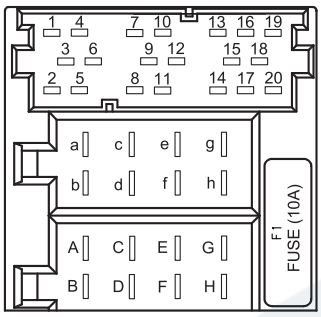

Figure 10: ISO connector pinout

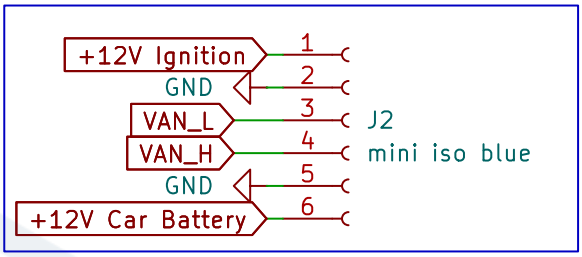

Figure 11: JST Connector in the schematic

VAN DATA	⇔	VAN_L
VAN DATA/	⇔	VAN_H
CD A/C GND	⇔	GND
CD A/C B/U	⇔	+12V Car Battery
+VAN	⇔	+12V Ignition

Table 2: ISO to JST connection legend

3.4 ESP32 Flashing

Flashing firmware on the ESP32 is done using UART. In order to flash the firmware, the ESP must first be put into flashing mode. This can be done by pulling pins `GPIO00` and `GPIO02` low when powering on the board. This can be achived in two ways on our board: Manually by pressing the `BOOT` momentary button while resetting the board with the `EN` button, or by connecting the `DTR` and `RTS` signals of the UART adapter to the apropriate pins on the JST programming connector. They are connected internally to the `EN` and `GPIO00` inputs and the ESP programmer (`esptool.py`) supports resetting and flashing the board using those signals.

The UART programming interface is exposed on another JST GH 6 pin connector as well as the `DTR` and `RTS` signals.

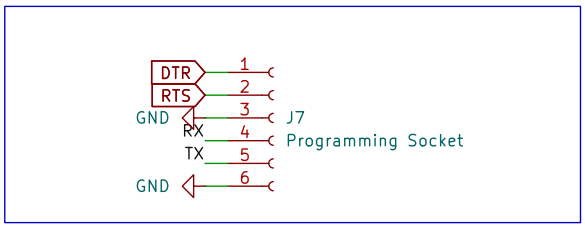

Figure 12: Programming header pinout

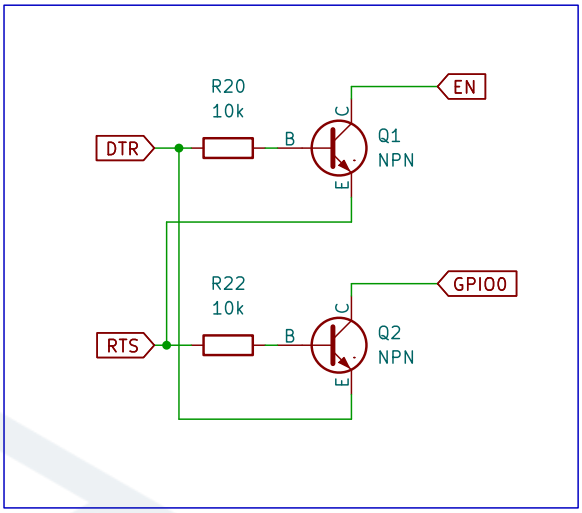

Figure 13: DTR and RTS signals

3.5 Power

TODO

3.6 Raspberry Pi

TODO

3.7 Misc

RTC, voltage divider voltmeter

4 Gateway module software

The gateway module is the most important part of this project as it interfaces with the car and is resposible for the entire system's power management. The module receives and parses VAN frames, organizes the data and then forwards it to the infotainment computer. It also does the reverse. It can receive data from the infotainment computer and transmit the appropriate VAN frame on the bus. The power management is also an important part of the software, as excessive idle power draw will drain the car's battery fairly quickly.

4.1 Framework and approach

For the software, we are using the ESP-IDF framework from Espressif. The framework is based on the FreeRTOS Operating System. Due to the nature of the OS and the software environment, the module software was written with an event-based approach.

We use the following events as a base for the main loops/tasks:

- Receiving VAN frames
- Receiving UART data from the computer
- Car state change
- Timer events
- Main task

Since the ESP32 has two cores, we delegated one core to exclusively handle receiving and parsing VAN frames, since they are very frequent and we don't want to miss any.

4.2 Receiving VAN frames

To receive VAN frames we have to manually read and time the RX pin states. Luckily, the ESP32 has a hardware peripheral that does just that: the RMT peripheral. It can monitor GPIO pin states and also measure how long it stayed in that state. Once we set the thresholds for the minimum and maximum signal lengths we can have the peripheral trigger an interrupt every time it sees an **EOF** sequence.

```
1  rmt_rx_channel_config_t rx_chan_config = {0};
2
3  ...
4
5  timeslot_interval_ns = 8 * 1000; // At 125 kTS/s
6
7  // EOF is 10 TS long
8  rx_recv_config.signal_range_max_ns = 10 * timeslot_interval_ns;
9  rx_recv_config.signal_range_min_ns = timeslot_interval_ns / 2;
10
11 ...
```

Listing 1: RMT timing initialization

Once the **EOF** sequence is found, the RMT peripheral driver triggers an interrupt. Inside that interrupt, we put a new event in the VAN Receiver task queue and restart the peripheral. The

VAN Receiver task takes the received RMT data from its queue and parses it. The data is an array of structs whose defition can be found in Listing 2. We reconstruct the VAN frame bit by bit from that array, taking into account the Manchester bits. After checking the checksum, we separate the identifier and data, put it in a new buffer and post it to the queue of the Main task for further processing.

```c
/**
 * @brief The layout of RMT symbol stored in memory, which is decided by the hardware design
 */
typedef union {
    struct {
        uint16_t duration0 : 15; /*!< Duration of level0 */
        uint16_t level0 : 1;     /*!< Level of the first part */
        uint16_t duration1 : 15; /*!< Duration of level1 */
        uint16_t level1 : 1;     /*!< Level of the second part */
    };
    uint32_t val; /*!< Equivalent unsigned value for the RMT symbol */
} rmt_symbol_word_t;
```

Listing 2: Definiton of the RMT data symbol in ESP-IDF

4.3 Parsing VAN frames

Once the VAN receiver task posts a new event, the Main task then calls the packet parser which checks the identifier and, if it's recognized, parses the data into global buffers and posts appropriate events to the Main queue depending on the packet being parsed. See listing 3 for an example VAN frame struct.

```c
/**
 * @brief VAN rpm and speed data struct. Iden 0x824
 *
 */
struct psa_van_rpm_speed_struct
{
    uint16_t rpm;           // RPM in big endian. Divide by 8 to get actual rpm;
    uint16_t speed;         // Speed in big endian. Divide by 100 to get actual speed;
    uint16_t distance;      // Distance covered. Increments with each passed decimeter
    uint8_t packet_changed; // Increments each time information significantly changes
} __attribute__((packed));
```

Listing 3: Example VAN frame struct.

4.4 Sending VAN frames

We use the TSS463C to send VAN frames to the car. The TSS is effectively additional 128 bytes of RAM that we have to manage, since we have to manage all the memory used for sending and receiving data. For now, the TSS is only used to send CD Changer packets so we can emulate it, and to send trip reset requests. The CD Changer messages are immediate reply messages (see 2.2.1), and the trip reset is a normal frame with an ACK request. The CD Changer packets are on a timer and sent every second, otherwise the onboard radio will think the CD Changer is absent and disable it.

The TSS communicates over SPI and is a bit cumbersome to use, so a custom driver was written to make it easier to use. Additionally, there is a seperate task that monitors the TSS message states and queues them to be sent.

4.5 UART communication

The infotainment computer communicates with the ESP32 over UART. The reason we use UART is its simplicity of use. The ESP mostly sends data to the computer, but can also receive requests from the computer.
The uart code lives in its own task that receives events from other tasks and can also send events to the main task. When the Main loop sees a change to the global data buffers (a VAN frame arrived, battery voltage changed, etc…) it posts an appropriate event to the uart task with the data to send to the computer. The protocol and data structs are custom and inspired by the MSP protocol used in Flight Controller firmware on FPV drones.

4.6 Event handling and tasks

The firmware uses several tasks that run as their own loops and wait for events from multiple queues. There are a total of 4 global queues in the firmware, one for each major component:

- Main Queue - Read by the Main task. All other tasks post their events to the Main task queue. The Main task then determines what to do and where to forward that event next.
- TSS Queue - Read by the TSS task. Used to queue packets for transmission by the TSS.
- UART Queue - Read by the UART task. Used to send packets to the Infotainment computer.
- VAN Queue (Unused) - Read by the VAN RMT task. Left for future use.

Additionally, there is an internal queue in the VAN RMT task. This queue is used exclusively between the RMT peripheral's interrupt and the VAN RMT task.

The events are defined as a struct (Listing 4) with the event identifier (Listing 5) and a pointer to some data. The data depends on the event being received.

4.7 Handling power states

It is important we manage the system's power consumption. To make that easier, we have defined several 'car states'. See table 3 for a list of those states, and the systems behaviour. One important component mentioned in that table is ESP32's deep sleep mode. When placed in deep sleep, the ESP is practically turned off. The only thing running on the chip is a special section of RAM, the RTC peripheral and, if needed, the ULP coprocessor. For our usecase, we use the same ADC pin we use for battery voltage measurement to wake up the ESP. Due to the voltage ranges of the pin, we can't use simple GPIO wake up, but must use the ULP coprocessor to measure voltage and wake the ESP up when it increases past a certain threshold.

4.8 The ULP coprocessor

The ESP32 has a very low power coprocessor onboard. It has a very simple architecture and uses very little power. It is ideal to use for ADC measurement and waking up the ESP. It cannot be programmed directly in C, but has two other ways of writing code. The first one is

writing the assembly code in a seperate file and building it into the firmware binary. The other one is using predefined C macros and writing the program in an array inside the C program and loading it that way. We opted to use the second option as it's way simpler to use. See Listing 6 for the ULP code used in deep sleep.

Description	**Computer state**	**ESP32 state**
Car in sleep mode (switched 12V rail is off)	Turned Off (no power)	Deep sleep
Car off and locked, not in sleep mode	Turned Off	Running
Car off, not locked, not in sleep mode	If coming from states above, turned off, otherwise display off	Running
Car off, radio turned on	Turned on, display on	Running
Car off, key in ACC	Turned on, display on	Running
Car off, key in IGN	Turned on, display on	Running
Car cranking	Turned on, display off	Running
Engine running	Turned on, display on	Running

Table 3: Car state table

```
struct mive_global_event
{
    enum mive_global_event_t event;
    void* ev_data;
};
```

Listing 4: Event struct definition

```
enum mive_global_event_t
{
    MIVE_EVENT_NONE = 0,
    MIVE_EVENT_GLOBAL_SLEEP,
    MIVE_EVENT_RMT_NEW_VAN_FRAME,
    MIVE_EVENT_TSS_RESET,
    MIVE_EVENT_TSS_ACTIVATE,
    MIVE_EVENT_TSS_IDLE,
    MIVE_EVENT_TSS_SLEEP,
    MIVE_EVENT_TSS_WRITE_FRAME,
    MIVE_EVENT_TSS_MONITOR_CHANNEL,

    MIVE_EVENT_VAN_UPDATE_AUDIO_MENU,
    MIVE_EVENT_VAN_NEXT_AUDIO_MENU_ITEM,
    MIVE_EVENT_VAN_CLOSE_AUDIO_MENU,

    MIVE_EVENT_UART_NEW_DATA, // New VAN Data to send
    MIVE_EVENT_UART_RECEIVE, // Received data from UART
    MIVE_EVENT_UART_SEND, // Send data to UART

    MIVE_EVENT_TIMER_UART_SEND,
    MIVE_EVENT_TIMER_1MS,
    MIVE_EVENT_TIMER_1S,

    MIVE_EVENT_ADC,
    MIVE_EVENT_STATE_CHANGE,
    MIVE_EVENT_POWER,
};
```

Listing 5: List of some events present in the firmware

```
const ulp_insn_t program[] = {
    I_MOVI(R0, 0),                  // Set reg. R0 to initial 0
    I_MOVI(R2, 0),                  // Set reg. R2 to initial 0
    M_LABEL(1),                     // LABEL 1 - Start of measurement
    I_ADDI(R0, R0, 1),              // Increment cycle counter (reg. R0)
    I_ADC(R1, PSA_ADC_UNIT, PSA_ADC_CHANNEL),           // Read ADC value to R1
    I_ADDR(R2, R2, R1),             // Add ADC value from R1 to R2
    M_BL(1, 4),                     // If cycle counter is less than 4, go to LABEL 1
    I_RSHI(R0, R2, 2),              // Divide accumulated ADC value in R2 by 4 and save it to R0
    M_BGE(2, high_adc_treshold),    // If average ADC value from reg.
                                    // R0 is higher or equal than high_adc_treshold, go to LABEL 2
    I_HALT(),                       // Halt the coprocessor, it will be woken up by the timer again
    M_LABEL(2),                     // LABEL 2
    I_WAKE(),                       // Wake up ESP32
    I_END(),                        // Stop ULP program timer
    I_HALT(),                       // Halt the coprocessor
};

size_t size = sizeof(program)/sizeof(ulp_insn_t);
ESP_ERROR_CHECK(ulp_process_macros_and_load(0, program, &size));
ulp_set_wakeup_period(0, 20000); // ULP wakeup timer
err = ulp_run(0);
```

Listing 6: ULP code that runs in deep sleep.

```
int ret = xQueueReceive(main_queue, &event, pdMS_TO_TICKS(1000));

if(ret == pdPASS)
{
    switch (event.event)
    {
    case MIVE_EVENT_RMT_NEW_VAN_FRAME:
        van_packet = (mive_van_packet_t*)event.ev_data;

        ret = psa_parse_van_packet(
            van_packet->iden,
            van_packet->packet_size,
            van_packet->packet,
            &global_libpsa_buffers);

        ...
        break;
    }
}
```

Listing 7: Example event handling

5 Infotainment computer software

The infotainment computer is a Raspberry Pi 4 with a DSI touchscreen display. It's running a custom Linux distribution built with the Yocto build system. The graphical application is written using C++ and the Qt framework. This software is not the focus of this assignment and will not be covered in depth.

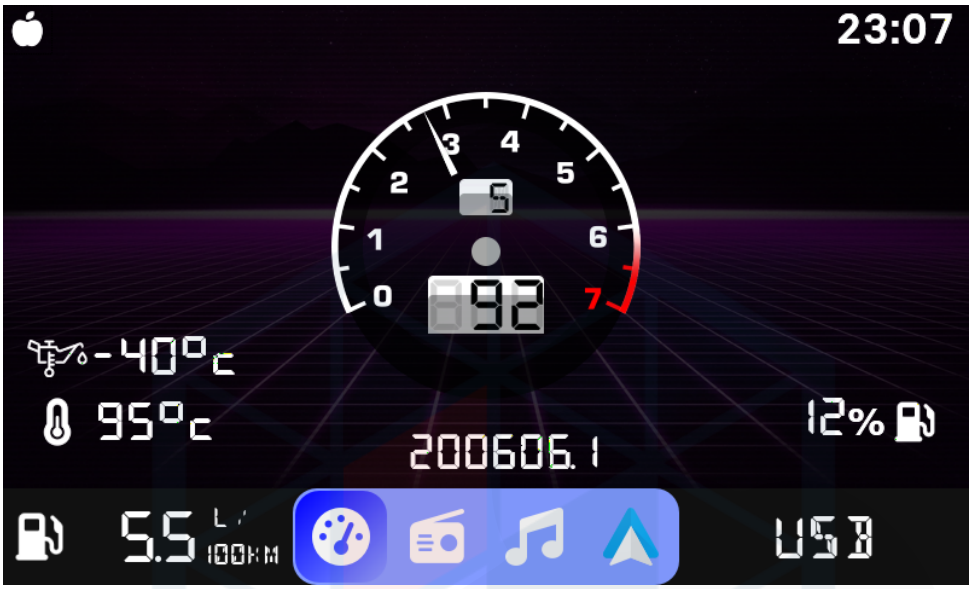

Figure 14: Screenshot of an early version of the software.

Features:

- Basic live telemetry. (Engine RPM and temperature, vehicle speed, fuel status)
- Vehicle information. (Door status, trip computer)
- Radio information. (Station information, CD player information)
- Local music player.
- Bluetooth player.